

Revisão ENADE — Banco de Dados

Do modelo relacional ao NoSQL

Tópicos Especiais II — ENADE · CC e SI · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Como funciona esta revisão

1. Recaptura relâmpago

- Os conceitos-chave de **cada unidade**, em uma passada rápida.
- Foco no que o **ENADE cobra**.

2. 🔥 Esquenta

- Questões **estilo ENADE** (situação-problema → A–D).
- Responda **antes** do gabarito comentado.

💡 A prova cobra **análise**, não decoreba: leia o contexto, elimine distratores, decida.

Parte 1

Recaptura relâmpago por unidade

Arquitetura ANSI-SPARC (3 níveis)

- **Externo (views):** o que **cada usuário/app** enxerga — recortes dos dados.
- **Conceitual (lógico):** o **esquema global** — tabelas, colunas, chaves, relacionamentos, regras.
- **Interno (físico):** como está **gravado** — arquivos, páginas, **índices**, ordenação.

Independência de dados

- **Física:** mudar índices/armazenamento **sem** mexer no conceitual nem nas apps.
- **Lógica:** mudar o **conceitual** (ex.: nova coluna) sem quebrar as **views** — mais difícil de garantir.

💡 ENADE: "reorganizou físico/índices sem afetar apps" = independência **física**.

ANSI-SPARC — visão em camadas

NÍVEL EXTERNO

Views — o recorte que cada usuário/app enxerga

↕ *independência lógica*

NÍVEL CONCEITUAL

Esquema lógico global — tabelas, chaves, relacionamentos

↕ *independência física*

NÍVEL INTERNO

Físico — arquivos, páginas, índices, ordenação

Modelo Relacional (Codd)

- Dados em **relações (tabelas)**: linha = **tupla**, coluna = **atributo**, valores válidos = **domínio**.
- **Chaves**: *superchave* (identifica) $\supseteq$ *candidata* (mínima) → uma vira **primária (PK)**, as outras **alternativas**; **estrangeira (FK)** referencia a PK de outra.

As 3 integridades

- **Entidade**: PK **não nula** e única.
- **Referencial**: FK **existe** na tabela referenciada **ou** é nula.
- **Domínio**: valor dentro do tipo/regra da coluna.



Violar integridade referencial = FK apontando para uma linha que não existe.

Álgebra relacional (a base do SELECT)

- σ seleção → **filtra linhas** (condição).
- π projeção → **escolhe colunas** (elimina duplicatas).
- $\bowtie$ junção → combina tabelas pela condição/chave.
- $\times$ produto cartesiano → **todas** as combinações.
- $\cup$ união · $\cap$ interseção · $-$ diferença (exigem tabelas **compatíveis**).
- $\div$ divisão → ideia de "**para todos**" (ex.: quem cursou **todas** as disciplinas).

💡 Ordem típica: junta ($\bowtie$) → filtra (σ) → projeta (π). **σ = linhas, π = colunas.**

MER e mapeamento para tabelas

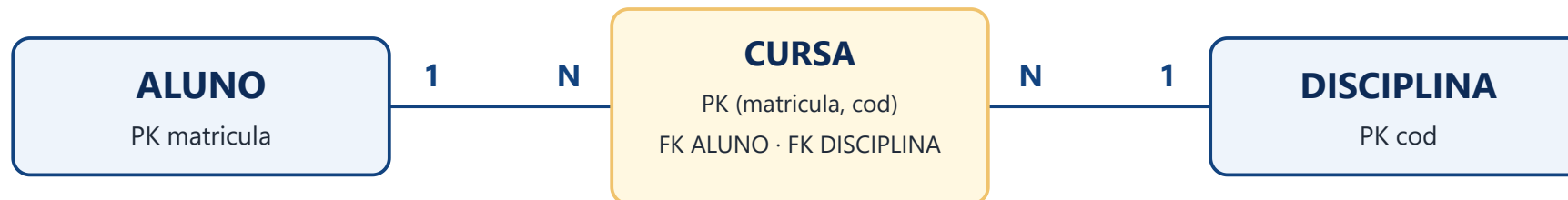
- **Atributos:** simples, **composto**, **multivalorado**, **derivado**.
- **Relacionamento + cardinalidade** (1:1, 1:N, N:N).
- **Generalização/especialização:** total×parcial, disjunta×sobreposta.
- **Entidade associativa** resolve N:N.

Regras MER → relacional

- **1:N:** a chave do lado **1** vira **FK** no lado **N**.
- **N:N:** cria **tabela associativa** com as duas FKs (**PK composta**).
- **1:1:** FK no lado de **participação total**.

💡 A "tabela associativa" do N:N é exatamente o que cai na modelagem da C1.

Mapeamento N:N — a tabela associativa



O N:N vira duas relações 1:N ligadas pela tabela associativa (PK composta = as 2 FKs)

Normalização — dependências funcionais

- **DF** $X \rightarrow Y$: o valor de X **determina** Y (ex.: $\text{CPF} \rightarrow \text{Nome}$); X é o **determinante**.
- **Parcial**: não-chave depende de **parte** de chave composta.
- **Transitiva**: não-chave depende de **outro não-chave**.
- **1FN**: atributos **atômicos**.
- **2FN**: 1FN + **sem dependência parcial**.
- **3FN**: 2FN + **sem dependência transitiva**.
- **BCNF**: todo **determinante** é chave candidata.

💡 **Parcial $\rightarrow$ 2FN · Transitiva $\rightarrow$ 3FN.**

Normalização — exemplo passo a passo

Desnormalizada: `Matricula(Aluno, Disciplina, Professor, Depto_Prof)` — chave `(Aluno, Disciplina)`; regras `Disciplina → Professor` e `Professor → Depto_Prof`.

- **Parcial:** `Professor` depende só de `Disciplina` (parte da chave) → fere a **2FN**.
- **Transitiva:** `Depto_Prof` depende de `Professor` (não-chave) → fere a **3FN**.

Resultado em 3FN

- `Matricula(Aluno, Disciplina)`
- `Turma(Disciplina, Professor)`
- `Professor_Info(Professor, Depto_Prof)`

⚠ Sem normalizar, o depto se repete em toda matrícula → anomalias de inserção, atualização e exclusão.

Normalização — a decomposição (visual)

Matricula(Aluno, Disciplina, Professor, Depto_Prof)

1 tabela — com dependência parcial e transitiva

decompõe até a 3FN ↓

Matricula

(Aluno, Disciplina)

Turma

(Disciplina, Professor)

Professor_Info

(Professor, Depto_Prof)

cada fato num só lugar → sem anomalias de inserção/atualização/exclusão

SQL — o que cai e a ordem lógica

- **DDL** CREATE/ALTER/DROP · **DML** INSERT/UPDATE/DELETE · **DQL** SELECT .
- **Ordem lógica:** FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY .
- **WHERE** filtra **linhas** (antes de agrupar); **HAVING** filtra **grupos** (aceita SUM/COUNT).

JOINS

- **INNER:** só o que **casa** nos dois lados.
- **LEFT:** **tudo** da esquerda (+ NULLs à direita).
- **FULL:** tudo dos **dois** lados.

💡 "Total do grupo passa de X" = `GROUP BY` + `HAVING` — nunca `WHERE SUM(...)`.

SQL — índices, views e FK

- **Índice:** acelera busca **seletiva** (poucas linhas). Em coluna de **baixa seletividade** (ex.: booleana que traz metade), o otimizador prefere **varredura** — o índice só onera escrita/espço.
- **View:** consulta nomeada (tabela virtual) — abstrai e protege o esquema.

FK — ações ao excluir o pai

- **CASCADE** : apaga os **filhos** junto.
- **RESTRICT/NO ACTION** : **bloqueia** a exclusão.
- **SET NULL** : deixa a FK **órfã** (nula).

Transações — ACID e níveis de isolamento

- **ACID**: **A**tomicidade · **C**onsistência · **I**solamento · **D**urabilidade.

Nível de isolamento	Dirty read	Non-repeatable	Phantom
READ UNCOMMITTED	pode	pode	pode
READ COMMITTED	não	pode	pode
REPEATABLE READ	não	não	pode
SERIALIZABLE	não	não	não

💡 Mais isolamento → menos anomalia, porém **menos concorrência** (mais bloqueio).

Em miúdos — Transações e isolamento

Pense numa **transferência bancária** : tirar R\$100 de você e creditar no outro — ou os **dois** passos acontecem, ou **nenhum**.

ACID no dia a dia

-  **Atomicidade:** tudo ou nada — nunca some dinheiro no meio.
-  **Consistência:** as regras nunca são quebradas (o saldo não fica inválido).
-  **Isolamento:** mil pessoas mexendo juntas, cada uma "sente" que está sozinha.
-  **Durabilidade:** confirmou = gravado pra sempre, mesmo faltando luz (é o comprovante).

 **Níveis de isolamento** = o quanto uma transação enxerga o *rascunho* da outra (como ler um texto que alguém ainda está editando). **Mais isolamento → menos surpresa, porém mais lento.**

Concorrência e recuperação

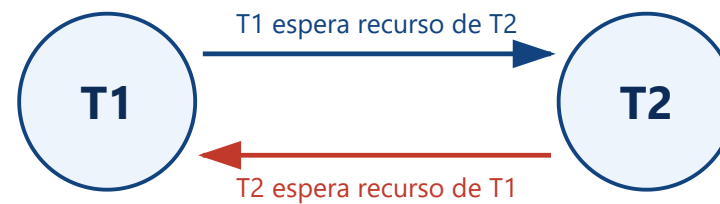
2PL (bloqueio em 2 fases)

- **Expansão** (só adquire locks) → **encolhimento** (só libera).
- Garante **serialização**, mas **não evita deadlock**.
- **Deadlock**: espera circular → o SGBD **detecta**, escolhe **vítima** e faz **rollback**. Prevenção por timestamp: *wait-die* / *wound-wait*.

Recuperação (WAL)

- Grava o **log antes** do dado.
- Confirmadas → **REDO** (durabilidade).
- Não-confirmadas → **UNDO** (atomicidade).
- **Checkpoint** limita o retrocesso do REDO.

Deadlock — o ciclo de espera (visual)



ciclo $\Rightarrow$ deadlock $\rightarrow$ o SGBD escolhe vítima + rollback

Em miúdos — Concorrência e recuperação

 **Bloqueio (lock)** = a plaquinha "**ocupado**" do **banheiro**: enquanto uma transação usa o dado, as outras **esperam**.

 **Deadlock (abraço mortal)** = duas pessoas, cada uma com **uma** chave que a outra precisa. Ninguém larga → o banco escolhe uma "**vítima**", desfaz o que ela fez e a outra segue.

 **Recuperação (WAL)** = o garçom **anota o pedido no bloquinho antes de ir à cozinha**. Se cair a energia, o banco lê o "bloquinho" e:

- **REFAZ** o que já estava confirmado;
- **DESAZ** o que estava pela metade.

 Por isso, mesmo caindo a luz no meio de tudo, o banco **nunca fica pela metade**.

NoSQL, CAP e BASE

4 famílias (com uso típico)

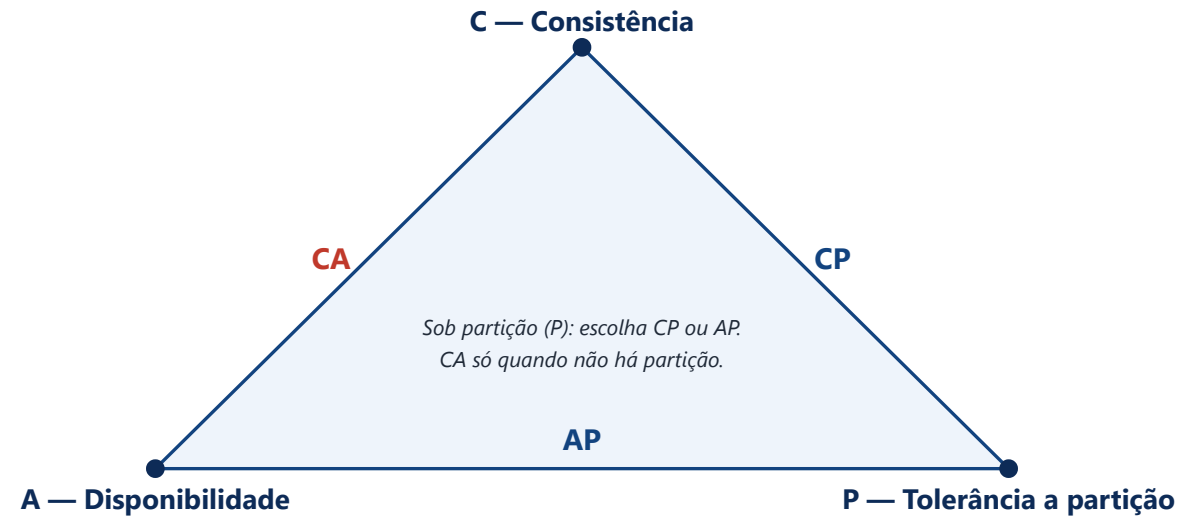
- **Chave-valor:** cache, sessão.
- **Documento:** JSON, catálogo.
- **Coluna:** analítico, escrita massiva.
- **Grafo:** relações, redes sociais.

CAP — sob **partição**, escolha **uma**

- **CP:** consistência, **recusa** em partição.
- **AP:** disponibilidade, **consistência eventual**.
- **CA:** só sem partição (irreal em rede distribuída).

💡 Relacional = **ACID** (forte); muitos NoSQL = **BASE** (disponível, consistência eventual).

CAP — escolha 2 dos 3



Em miúdos 🧑 — NoSQL, CAP e BASE



📁 **CAP** = duas **filiais** que perdem contato:

- **CP**: "**não vendo** sem confirmar" — nunca erra, mas **fica indisponível**.
- **AP**: "**vendo e acerto depois**" — nunca para, mas por um tempo os números **divergem**.

💬 **BASE** = um **grupo de WhatsApp**: nem todos recebem no mesmo instante, mas em pouco tempo **todos veem a mesma coisa** (consistência eventual).

👉 Regra de bolso: **dinheiro / estoque crítico** → **ACID** · **feed, curtidas, carrinho que não pode cair** → **BASE**.

Parte 2 — Esquenta

Banco de Dados · questões estilo ENADE (A–D)

Responda antes de virar o slide!

Q1 — Álgebra relacional

Uma secretaria acadêmica usa `Aluno(Matricula, Nome, IdCurso)` e `Curso(IdCurso, NomeCurso)`. É preciso emitir uma lista com **somente os nomes** dos alunos do curso de **Sistemas de Informação**.

Assinale a expressão de álgebra relacional correta.

- A) $\pi_{\text{Nome}}(\sigma_{\text{NomeCurso}='Sistemas de Informação'}(\text{Aluno} \bowtie \text{Curso}))$
- B) $\pi_{\text{Nome}}(\text{Aluno} \bowtie \text{Curso})$
- C) $\sigma_{\text{NomeCurso}='Sistemas de Informação'}(\text{Aluno} \bowtie \text{Curso})$
- D) $\pi_{\text{NomeCurso}}(\sigma_{\text{Nome}='Sistemas de Informação'}(\text{Aluno} \bowtie \text{Curso}))$

Q1 — Gabarito: A

Relembrando: σ filtra **linhas**; π escolhe **colunas**; a junção casa `IdCurso`.

- Precisa **filtrar** pelo curso (σ) e **projetar** só o `Nome` (π). 
- **B** não filtra (traz todos); **C** filtra mas **não projeta** (devolve a tupla inteira); **D** usa atributos trocados.

Q2 — SQL com agregação

Em `Cliente(id, nome)`, `Pedido(id, id_cliente)`, `Item(id_pedido, qtd, valor)`, o marketing quer os **clientes cujo total gasto** ($\sum \text{qtd} * \text{valor}$) **passa de R\$ 1.000**.

Qual consulta atende?

- A) `WHERE SUM(qtd*valor) > 1000` com `GROUP BY nome`.
- B) junta as três tabelas, agrupa por cliente e usa `HAVING SUM(qtd*valor) > 1000`.
- C) agrupa por cliente e usa `HAVING qtd*valor > 1000`.
- D) `WHERE valor > 1000`, item a item.

Q2 — Gabarito: B

Relembrando: filtro sobre **agregado por grupo** → `GROUP BY` + `HAVING` (o `WHERE` não aceita `SUM`).

- **A** usa `SUM` no `WHERE` (inválido); **C** compara item a item, não o total; **D** filtra itens caros, não o **gasto total** do cliente.  = **B**.

Q3 — Normalização

Um analista modela $R(\text{Aluno}, \text{Disciplina}, \text{Professor})$ com chave $(\text{Aluno}, \text{Disciplina})$ e a regra $\text{Disciplina} \rightarrow \text{Professor}$. Ele nota **redundância**: o professor se repete em cada matrícula.

A relação **viola** a:

- A) 1FN, por atributo multivalorado.
- B) 3FN, por dependência transitiva.
- C) 2FN, por dependência parcial de Professor em relação à chave.
- D) Nenhuma; já está na 3FN.

Q3 — Gabarito: C

Relembrando: Professor depende só de Disciplina (parte da chave composta) → dependência parcial → fere a 2FN. ✓

- Não é transitiva (3FN) nem atomicidade (1FN). **Correção:** Turma(Disciplina, Professor) + Matricula(Aluno, Disciplina).

Q4 — Mapeamento de generalização

`Pessoa` especializa-se em `PessoaFisica` (CPF) e `PessoaJuridica` (CNPJ), de forma **total e disjunta**. Deseja-se **evitar colunas nulas** e impedir que um cadastro seja PF e PJ.

Qual estratégia é a mais adequada?

- A) Uma tabela `Pessoa` com CPF e CNPJ (a maioria nula).
- B) Uma **tabela por subtipo**, cada uma com os atributos comuns.
- C) `Pessoa` + subtabelas, permitindo ser PF **e** PJ.
- D) `Pessoa` com listas de CPF/CNPJ por vírgula.

Q4 — Gabarito: B

Relembrando: especialização **total + disjunta** → **uma tabela por subtipo** elimina nulos e garante a exclusividade. 

- **A** gera muitos nulos; **C** permitiria sobreposição; **D** fere a 1FN.

Q5 — Isolamento

No sistema bancário, T1 lê o saldo; antes de T1 terminar, T2 **credita e confirma**; ao **reler**, T1 obtém **valor diferente**.

A anomalia é:

- A) Leitura suja (*dirty read*).
- B) Perda de atualização (*lost update*).
- C) Leitura fantasma (*phantom*).
- D) Leitura não repetível (*non-repeatable read*).

Q5 — Gabarito: D

Relembrando: reler a **mesma linha** e achar valor diferente (por *update confirmado* no meio) = **leitura não repetível**. 

- *Dirty* = ler não-confirmado; *phantom* = novas linhas; *lost update* = escritas se sobrepõem. Evita-se com `REPEATABLE READ`.

Q6 — Concorrência

Com 2PL, T1 trava A e T2 trava B; então T1 pede B e T2 pede A, ambas travadas.

Assinale a correta.

- A) O 2PL garante que deadlock nunca ocorra.
- B) Ocorre **deadlock**; o SGBD detecta, escolhe vítima e faz rollback.
- C) As duas concluem sem espera.
- D) É uma leitura fantasma.

Q6 — Gabarito: B

Relembrando: 2PL garante **serialização**, mas **não evita deadlock**. Espera circular → o SGBD **aborta uma vítima** (rollback) e a outra segue. 

 Prevenção por timestamp: *wait-die* (esperar-morrer) / *wound-wait* (ferir-esperar).

Q7 — Recuperação (WAL)

Após falha de energia: T1 deu commit **antes** da falha (páginas talvez não gravadas); T2 estava **ativa** (sem commit).

A recuperação aplica:

- A) UNDO em T1, REDO em T2.
- B) REDO em ambas.
- C) REDO em T1, UNDO em T2.
- D) Nada; o commit de T1 dispensa recuperação.

Q7 — Gabarito: C

Relembrando (WAL): confirmadas → **REDO** (durabilidade, caso de T1); não-confirmadas → **UNDO** (atomicidade, caso de T2). 

- O **checkpoint** limita o quanto o REDO precisa retroceder.

Q8 — Índices

Sobre uma tabela de **milhões de linhas**, a equipe decide onde criar índices. Em qual caso o índice **tende a NÃO ajudar**?

- A) Coluna `cpf` única em `WHERE cpf = ...`.
- B) FK usada em `JOIN`.
- C) Coluna **booleana** cujo filtro retorna **metade** das linhas.
- D) Coluna de data em faixas (`BETWEEN`).

Q8 — Gabarito: C

Relembrando: índice compensa quando a busca é **seletiva**. Coluna de **baixa seletividade** que retorna metade da tabela → o otimizador prefere **varredura completa**; o índice só onera escrita/espço. 

Q9 — NoSQL e CAP

Um **carrinho de compras global** precisa ficar **disponível e aceitando escritas** mesmo com **partições de rede**, tolerando inconsistência **temporária**.

Segundo o CAP, a escolha adequada é:

- A) **AP** com consistência eventual (ex.: banco de documentos distribuído).
- B) **CA** garantindo as três propriedades.
- C) Relacional com bloqueio global síncrono.
- D) **CP**, recusando escritas durante a partição.

Q9 — Gabarito: A

Relembrando (CAP): sob **partição**, escolhe-se **C ou A**. O carrinho quer **disponibilidade + escrita** → **AP** (modelo **BASE**, consistência eventual). 

- **B** é impossível sob partição; **D** recusaria escritas; **C** não escala.

Q10 — Arquitetura

A equipe **reorganiza o armazenamento físico** e **cria índices** de uma tabela, **sem alterar** aplicações nem o esquema conceitual.

Na ANSI-SPARC, isso é:

- A) Independência lógica.
- B) Independência física.
- C) Ausência de independência.
- D) Independência referencial.

Q10 — Gabarito: B

Relembrando: mudar o **nível interno** (físico/índices) sem afetar o conceitual/aplicações = **independência física**. 

- A **lógica** seria mudar o **esquema conceitual** sem afetar as **views** — não é o caso.

Fim da revisão de Banco de Dados

Próximo: mais Esquentas e o Aulão de BD (23/09).

Gabarito: 1A · 2B · 3C · 4B · 5D · 6B · 7C · 8C · 9A · 10B

 **Aulão (23/09):** resolveremos questões em **grupos** — traga suas dúvidas das unidades acima.